08 · Hak Akses Data (RLS)

Row Level Security (RLS) membuat setiap pengguna hanya melihat baris data yang menjadi bagiannya, otomatis di semua chart, dashboard, SQL Lab, Studio, Asisten AI, dan pilihan filter. Ada dua cara; boleh dipakai bersamaan.

Cara	Cocok untuk	Diatur di
A. Aturan per role	semua perusahaan; jumlah kelompok sedikit (cabang, divisi)	Settings → Row Level Security
B. Otomatis dari profil pengguna	organisasi besar dengan tabel profil karyawan (wilayah, cabang, jabatan)	`config\akses.yaml` + `{{ sps_akses(...) }}`

A. Aturan per role

1. Buat role per kelompok, mis. `Cabang Jakarta` (**Settings → List Roles → +**, salin izin `Gamma`).
2. Masukkan pengguna ke role itu (**Settings → List Users**).
3. **Settings → Row Level Security → + Rule**:

Isian	Contoh
Rule name	`akses__latihan_penjualan__jakarta`
Filter type	**Regular**: berlaku hanya untuk role yang dipilih
Datasets	`latihan_penjualan` (bisa beberapa dataset sekaligus)
Roles	`Cabang Jakarta`
Group key	`cabang` (aturan dengan group key sama digabung dengan OR)
Clause	`nm_cabang = 'Cabang Jakarta'`

4. Uji dengan akun anggota role itu.

Jenis	Perilaku
Regular	syarat ditambahkan **hanya** untuk role terpilih; role lain melihat semua
Base	syarat ditambahkan untuk **semua kecuali** role terpilih. Pola aman: Base `1 = 0` (tidak melihat apa pun) + kecualikan role yang boleh melihat semua, lalu aturan Regular per kelompok

Ekstrak dan dataset hasil Persiapan data **mewarisi** aturan RLS sumbernya (13).

B. Otomatis dari profil pengguna (opsional)

Setiap orang hanya melihat data sesuai **profilnya** (mis. wilayah, cabang, jabatan), otomatis saat login. Profil dibaca dari database perusahaan (mis. tabel karyawan). **Bawaan: tidak aktif.** Aktif bila `profil` di `config\akses.yaml` diisi (nama koneksi + query).

Cara kerja

```
LOGIN (LDAP atau akun lokal)
  └─ sps_security.py → sps_akses.sinkron_profil(username)
       └─ jalankan profil.sql (akses.yaml) di koneksi profil.database, parameter :username
       └─ simpan ke tabel sps_profil_pengguna (metadb), format JSON
       └─ cocokkan ke aturan akses.yaml → lingkup + role berawalan "Akses "

SETIAP QUERY CHART / FILTER / SQL LAB
  └─ aturan RLS dataset:  {{ sps_akses(wilayah='KD_WILAYAH', cabang='KD_CABANG') }}
       └─ menjadi:  KD_WILAYAH IN ('W1') AND KD_CABANG IN ('C01')
```

Menyiapkan profil

1. Hubungkan database sumber profil: **Pusat Kendali → Hubungkan data** (akun hanya-baca), mis. bernama `HR Karyawan`.
2. Isi `config\akses.yaml`:

```
profil:
  database: "HR Karyawan"
  sql: |
    SELECT ID_KARYAWAN AS id, NAMA AS nama, KD_WILAYAH AS wilayah, KD_CABANG AS cabang,
           TIPE_KANTOR AS tipe_kantor, JABATAN AS jabatan
    FROM KARYAWAN WHERE ID_KARYAWAN = :username
```

Nama kolom hasil query (huruf kecil) menjadi **nama atribut** yang dipakai di aturan. Query wajib `SELECT` / `WITH` dan memakai `:username`; dijalankan maksimal `AKSES_TIMEOUT` detik (bawaan 8) agar login tidak menggantung. Sumber profil mati → profil terakhir yang tersimpan tetap dipakai.

Aturan

Contoh aturan	Syarat (`kapan`)	Lingkup data	Role
Kantor pusat	tipe_kantor = PUSAT	semua	Akses Pusat
Kepala wilayah	tipe_kantor = WILAYAH	wilayahnya	Akses Wilayah
Staf cabang	tipe_kantor = CABANG	wilayah **dan** cabangnya	Akses Cabang
(tidak cocok)	—	**tidak ada data**	—

```
aturan:
  - nama: Kantor pusat
    kapan: {tipe_kantor: PUSAT}
    lingkup: semua
    peran: ["Akses Pusat"]
  - nama: Staf cabang
    kapan: {tipe_kantor: CABANG}
    lingkup: [wilayah, cabang]
    peran: ["Akses Cabang"]
```

Aturan dibaca dari atas; yang **pertama cocok** dipakai. Satu aturan boleh menggabungkan **1 sampai banyak** atribut, baik di syarat maupun di lingkup. Pengecualian per orang (username login, menimpa aturan):

```
pengecualian:
  "budi.santoso": {lingkup: {wilayah: ["W1", "W2"]}, peran: ["Akses Wilayah"]}
  "siti.rahayu": {lingkup: semua, peran: ["Akses Pusat"]}
```

Role berawalan `awalan_peran` (bawaan `"Akses "`) dikelola otomatis: dicabut lalu diberikan ulang setiap login. Perubahan `akses.yaml` berlaku pada **login berikutnya**, tanpa restart.

Syarat pada data ("filter wajib")

Agar data bisa dibatasi, **dataset wajib punya kolom** untuk setiap atribut di lingkup pemakainya. Di dashboard as code, kolom itu didaftarkan di `akses`:

```
Dataset(
    key="penjualan_per_cabang",
    sql="SELECT KD_WILAYAH, KD_CABANG, ... FROM ...",
    columns=[...],
    akses={"wilayah": "KD_WILAYAH", "cabang": "KD_CABANG"},
)
```

Nilai `akses`	Arti
`{"cabang": "KD_CABANG", ...}`	Dibatasi sesuai profil (data per wilayah/cabang)
`{}` (bawaan)	Data **menyeluruh**: hanya lingkup `semua` (pusat & Admin)
`None`	Data referensi, tidak dibatasi (mis. pilihan tahun/bulan)

Builder otomatis membuat aturan RLS bernama `akses__<nama dataset>` (lihat **Settings → Row Level Security**). Dataset yang dibuat manual lewat UI: tambahkan aturan RLS sendiri dengan klausa yang sama.

Prinsip **gagal = tertutup**: kolom yang diwajibkan tidak ada, nilai profil kosong, profil tidak ditemukan, atau tidak ada aturan cocok → `1 = 0` (tidak ada data). Admin Superset selalu `1 = 1`.

Hak akses dashboard

Fitur `DASHBOARD_RBAC` aktif: dashboard bisa dibatasi per role lewat dashboard → **Edit properties → Roles**. Kosong = mengikuti akses dataset.

Hal penting

- **Role bawaan pengguna sebaiknya `Gamma`** (`LDAP_DEFAULT_ROLE` di `.env`). Role `Alpha` bisa membuat dataset baru langsung dari tabel sumber sehingga **melewati** hak akses. Berikan Alpha hanya ke analis tepercaya.
- SQL Lab hanya untuk Admin (atau role `sql_lab`). `RLS_IN_SQLLAB` aktif, tetapi query bebas tetap bisa membaca tabel yang tidak punya dataset ber-RLS.
- Melihat profil yang tersimpan (cara B): tabel `sps_profil_pengguna` di metadb (`docker compose exec metadb psql -U superset -d sps_superset -c "select * from sps_profil_pengguna limit 5"`).
- Log setiap login mencatat aturan yang dipakai: `ALDx akses budi.santoso: aturan 'Staf cabang', lingkup ['wilayah', 'cabang'], role [...]`.